AI chat summary: BrightPath support assignment

Candidate: Abhishek Sharma · **Role:** Forward Deployed Engineer, ClearTax
Tool: Cursor (AI coding agent) · **Date:** 25–26 Oct 2026
Live demo: brightpath-ai-support.vercel.app · **Code:** github.com/abhishek-sharma-2001/brightpath-ai-support

This summarises how the problem was worked through with the AI: the approach, what the analysis found, the decisions and trade-offs, and what was produced. The full recommendation is in `01_Solution_Memo.md`.

---

1. The brief given to the AI

The full problem statement (BrightPath's 30-person support team, the CEO's goal of 12 agents in six months with a demo next week, and the views of Support, Finance and Product, plus Exhibits A and B), with the instruction to complete the assignment in a folder on the Desktop.

2. How the problem was approached

Step 1: Don't start with the chatbot. Start with where agent time goes. The CEO framed the solution ("an AI chatbot"), so the first move was to test that framing against the data. Multiplying volume by handling time for each ticket type showed that **ticket share and time share are very different**:

- Course content questions are 38% of tickets but only 10% of agent time.
- Video, payment and refund tickets are 52% of tickets but **84% of agent time**.

That one table changes the conversation. A FAQ chatbot works in the part of the problem that costs the least.

Step 2: Read each stakeholder quote as a constraint or a hint.

Source	Read as
CEO: "12 people in six months, demo next week"	Target and deadline, to be tested rather than accepted
Support Lead: "most of our time goes into payments and refunds"	Hint, confirmed by the data (53% of time)
Head of Finance: "no refund without a human"	**Hard constraint.** Design around it, don't argue with it.
PM, quietly: "video player breaks on some Android phones"	**Hidden lever.** 78% of video tickets are Android.
Exhibit B: 60% of failed payments self-reverse, "but students aren't told"	**Hidden lever.** Thousands of tickets exist only because nobody tells the student.

Step 3: Model it instead of guessing. We built a small capacity model (`demo/capacity_model.py`) that:

- calibrates agent capacity to today (4,663 handling hours ÷ 30 agents = 155 hours per agent per month, about 88% occupancy), so no outside utilisation figure is needed;
- models each lever separately, with explicit, adjustable assumptions;
- runs three scenarios (FAQ chatbot only, conservative full plan, expected full plan), a lever-by-lever breakdown and a sensitivity table.

Step 4: Design the solution around the levers, not around a chatbot. Five workstreams came out of the analysis, ranked by impact: fix the Android bug; proactive payment alerts with live payment lookup; AI-prepared refunds with human approval; an assistant for content and login; video troubleshooting with automatic diagnostics.

Step 5: Build the demo the CEO asked for, but make it demonstrate the plan. Instead of a generic Q&A bot, the demo shows each lever working: device-aware handling of the Android bug, payment reversal explanations, auto-unlock for captured payments, refund cases prepared for Finance, proactive webhook alerts, a support queue where agents never re-ask, and the capacity model as a live tab.

Step 6: Verify everything. 24 automated end-to-end tests (every student scenario, the refund guardrails, the model's numbers, an HTTP smoke test), a browser walkthrough of every screen with screenshots, and a re-check of each figure quoted in the memo against the model output. Verification caught and fixed a few things: a layout issue, one percentage in the memo (content + login is 16% of time, not 15%), and the missing Hinglish support described below.

3. Key insights

1. **Tickets aren't workload.** A FAQ chatbot that perfectly answers every content and login question frees about 4.7 agents. Realistically, a chatbot alone gets the team from **30 to about 27**.
2. **The biggest lever is a bug fix, not AI.** The Android video bug costs about 4,836 tickets, 1,128 agent hours and **7.3 agents every month**, more than twice all content questions combined.
3. **About 5.3 agents spend their time saying "wait 48 hours".** A proactive SMS/WhatsApp triggered by the payment gateway removes most of these tickets, and it doesn't even need AI.
4. **Finance's rule doesn't block automation.** The AI does everything up to the decision (collect, verify, check policy, score fraud risk, recommend), and the human decides in about 10 minutes instead of 25. Fraud control gets stronger, because every case gets the same checks and an audit trail.
5. **12 is reachable but fragile.** Expected: 11; conservative: 17. If the Android fix slips, or refund review takes 15 minutes instead of 10, the answer is 13–14. So the recommendation is to commit to **12–14 by month 6** and tie every headcount step to service metrics.
6. **The team has no slack today** (about 88% occupancy). Cutting people before the automation is proven goes straight into backlog, refunds, chargebacks and bad reviews.

4. Decisions and trade-offs

Decision	Why	Trade-off
Rule-based intent detection by default, optional LLM	The demo runs offline, behaves the same every time in front of a CEO, and needs no API key	Less flexible language understanding. Production would use the LLM path with an evaluation set of real messages.
LLM only classifies intent; replies are built from tool data	No made-up class times, amounts or refund promises	Replies are less conversational. Production can let the LLM phrase replies limited to tool outputs.
Refund approval in a separate module the assistant never imports	Turns Finance's rule into something the code enforces, not just a policy statement	None for the demo. Production adds role-based access.
Python standard library only	Runs on any laptop with no install step	A plain-JavaScript frontend rather than a framework

Decision	Why	Trade-off
Agent capacity calibrated to the current team	Avoids inventing an occupancy figure	Assumes today's working patterns continue
Escalated tickets keep handling times near today's	The easy tickets get filtered out, so what's left is harder	More conservative than assuming diagnostics make every ticket faster
Every assumption is a slider in the Impact tab	The assumptions are the debatable part, so stakeholders can test them live	None
Basic Hinglish support ("paise kat gaye", "video nahi chal raha")	Indian exam-prep students often write this way	Keyword coverage is basic; production needs an LLM and evaluation in Hinglish and Hindi
Hosted demo runs the Python backend in the browser (Pyodide) on Vercel	Serverless functions don't keep in-memory state, and reviewers opening the link at the same time would share one demo. In the browser, everyone gets a private, fresh demo from the same code that the tests cover	A few seconds of loading on the first visit, and the hosted version is rules-only (no API key in the browser)

5. What was produced

Item	Description
`01_Solution_Memo.md`	Recommendation, numbers, workstreams, rollout and headcount path, guardrails, metrics, risks
Live demo	The prototype hosted on Vercel, so it can be opened from a link with nothing to install
`demo/`	Working prototype (student chat, support queue, finance approvals, insights, impact model) plus 24 tests
`demo/capacity_model.py`	Reproducible model behind every number in the memo
`screenshots/`	Screens from the demo walkthrough
`README.md`	How to run the demo and a five-minute demo script

6. Open questions for BrightPath

1. **Channel mix.** What share of the 25,000 tickets come by phone? A chat assistant can't deflect calls without an IVR or voice bot.
2. **Are the Exhibit A categories and handling times accurate?** I'd check them against two weeks of real tickets.
3. **Android fix.** What's the root cause, how much engineering effort does it need, and when can it ship? It's on the critical path for the headcount target.
4. **Payment gateway.** Which provider? Are "failed but debited" webhooks and a status API available?
5. **Refund policy.** Is there a written policy, and what did past fraud look like? (The demo uses a sample policy: full refund within 7 days and under 25% watched, 50% within 30 days and under 50% watched.)
6. **Seasonality.** How big are the exam-season peaks (admit cards, results)?
7. **Cost per agent.** Needed to express the plan in rupees, not just headcount.

7. Next steps with more time

- Run an LLM over a real ticket sample to check categories, handling times and true resolution rates before committing to headcount numbers.

- Switch intent detection to the LLM path and measure accuracy against a labelled set of real student messages, including Hindi and Hinglish.
- Integrate WhatsApp Business, since that's where Indian students already are, and an IVR that sends a chat link for phone callers.
- Connect a payment gateway sandbox and the real course schedule, replacing the mock data.
- Shadow the support team for a day to find failure cases that the averages hide.